



INTRODUÇÃO AO CLAUDE CODE

Introdução ao Claude Code *

*Um guia estruturado com os principais conceitos sobre claude code
— Feito a partir dos cursos e palestras da Anthropic.*

MATERIAL FEITO COM CLAUDE DESIGN A PARTIR DE ANOTAÇÕES

LUIZA REIXACH CASTRO

 [LINKEDIN.COM/IN/LUIZACASTRO02](https://www.linkedin.com/in/luizacastro02)

17 MAIO · 2026

MATERIAL DE ESTUDO
NÃO-OFICIAL

Sumário

01	O que é o Claude Code	04
	● Claude Code como assistente de código ● LLM, harness e tool calls ● Tokens, entrada, saída e custo de contexto ● Modelos da Anthropic e diferenças práticas ● Janela de contexto	
02	Primeiros passos	09
	● Iniciando com <code>/init</code> ● O que é o CLAUDE.md ● Tipos de CLAUDE.md e escopo ● Editando o CLAUDE.md ● Usando <code>@</code> para mencionar arquivos	
03	Planejamento e contexto	12
	● Planning mode e thinking ● Interromper e redirecionar com <code>Esc</code> ● Voltar para pontos anteriores com <code>/rewind</code> ● Compactar texto com <code>/compact</code> ● Começar do zero com <code>/clear</code>	
04	Comandos essenciais e personalizados	15
	● Comandos essenciais ● Atalhos <code>/</code>, <code>@</code>, <code>!</code>, <code>Shift+Tab</code> ● Comandos personalizados como prompts reutilizáveis ● Argumentos com <code>\$ARGUMENTS</code>	
05	Hooks	17
	● O que é um Hook ● Tool calls que podem disparar hooks ● PreToolUse e PostToolUse ● Exemplos práticos de automação	

06	Subagentes	20
	● O que são subagentes ● Como subagentes protegem a janela de contexto ● Como o Claude decide usar um subagente ● Criando e configurando subagentes ● Quando funcionam bem	
07	Skills	23
	● O que é uma skill ● Quando o Claude ativa uma skill ● Como criar uma skill (SKILL.md, descrição, gatilho) ● Ciclo de vida e progressive disclosure ● Quando a skill não é usada como esperado	
08	MCP Server	26
	● O que é Model Contexto Protocol ● Playwright MCP Server	
09	Como escolher o recurso certo	27
	● Mapa de decisão ● Matriz comparativa: CLAUDE.md, Slash Command, Hook, Agent, Skill ● Boas práticas ● Referências de estudo	

01 O QUE É O CLAUDE CODE

O que é o Claude Code.

— Claude Code como assistente de código

O Claude Code é um assistente de código baseado em LLMs, isto é, em *large language models*. De forma simples, ele é um sistema que permite usar modelos como Claude Sonnet, Claude Opus ou Claude Haiku para **realizar tarefas de programação dentro de um projeto real, de modo autônomo**.

A interação começa por meio de um **prompt**: você descreve o que quer, por exemplo, "arrumar um erro", "criar testes para este arquivo", "entender por que essa API falha" ou "adicionar uma nova funcionalidade". A partir dessa solicitação, o Claude Code interpreta o pedido, reúne contexto, escolhe ferramentas, analisa arquivos, formula um plano quando necessário e executa ações como editar código, rodar testes, procurar arquivos, abrir documentação ou verificar resultados.

Uma boa forma de visualizar esse fluxo é pensar no Claude Code como um **ciclo de trabalho**:

1. Você envia uma tarefa em linguagem natural.
2. O modelo entende a intenção e identifica o contexto necessário.
3. O Claude Code usa ferramentas para ler arquivos, buscar informações ou executar comandos.
4. O resultado dessas ferramentas volta para o modelo.
5. O modelo decide o próximo passo: responder, editar, testar, corrigir ou pedir mais contexto.
6. O ciclo continua **até a tarefa estar concluída ou até você interromper e redirecionar**.

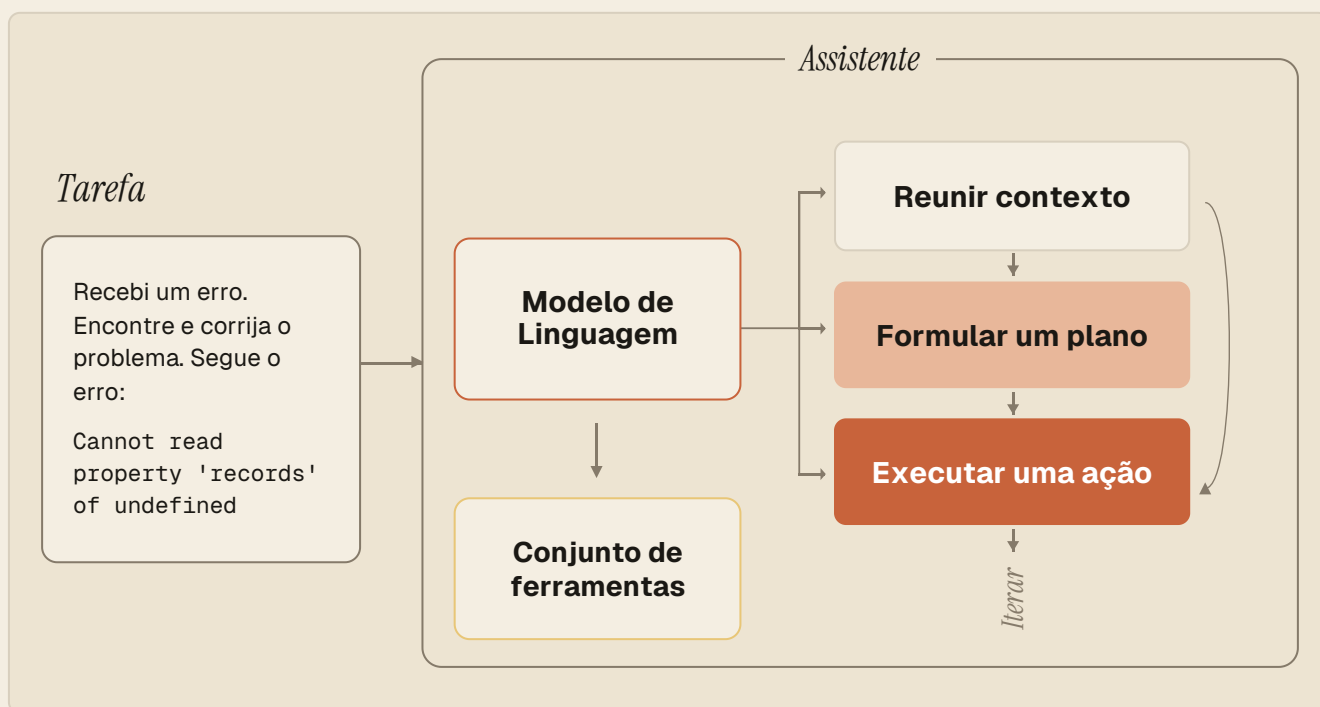
Esse ciclo é importante porque o Claude Code não é apenas um chatbot que responde perguntas sobre programação. Ele funciona como um **agente de desenvolvimento**: entende a tarefa, age sobre o ambiente e verifica o resultado.

O Claude Code pode ser instalado via CLI no terminal ou como extensão em IDEs, como VS Code. Ele está disponível a partir do plano Pro, que custa \$20 por mês. No final do documento estão disponíveis links para a instalação.

— Como o Claude Code funciona

O esquema abaixo representa o ciclo de um *agent loop*: uma tarefa entra, o modelo de linguagem decide o que fazer, aciona ferramentas e itera entre reunir contexto, formular um plano e executar ações até a tarefa estar concluída.

- A maioria dos assistentes de código usa modelos de linguagem hospedados remotamente.
- O Claude Code usa a família de modelos Claude hospedados na Anthropic, AWS ou Google Cloud (configurável).



— LLM, harness e tool calls

O ponto mais importante para iniciantes é separar duas coisas: a **LLM** e o **assistente de código**. A LLM, como Claude, GPT ou outros modelos, não executa ações por conta própria, seu papel é receber texto e retornar texto. Ela não pode, sozinha, abrir arquivos, editar código, rodar um comando no terminal ou consultar um banco de dados. Quando parece que está fazendo isso, na prática existe outra camada no meio: o assistente de código, também chamado de *harness*, *scaffolding* ou plataforma de execução.

A LLM

Trabalha *somente* com *texto*

O modelo raciocina e descreve o que *seria útil* fazer, mas não tem braços próprios. Sem uma camada externa, só consegue responder com palavras.

O HARNESS (CLAUDE CODE)

Quem realmente *age* sobre o ambiente.

Fornece acesso controlado a ferramentas: ler e editar arquivos, busca no projeto, execução de comandos, integrações externas. É ele que transforma raciocínio em ação concreta.

Isso acontece por meio de **tool calls**. Uma *tool call* é uma chamada estruturada. O modelo retorna uma estrutura dizendo, por exemplo, que deseja executar um comando Bash, ler um arquivo ou editar determinado trecho. O *scaffolding* interpreta essa chamada, executa a ação e devolve o resultado para o modelo:



Essa separação torna o sistema mais confiável. Sem ferramentas, a LLM só consegue responder com texto; com ferramentas, ela passa a poder agir: ler código, editar arquivos, rodar testes, procurar documentação e interagir com serviços conectados.

— Prompts e arquivos Markdown

Um **prompt** é a instrução que você envia ao modelo. Ele pode ser curto, como "corrija esse bug", ou mais completo, incluindo regras de negócio, caminhos de arquivos, critérios de sucesso, exemplos esperados e restrições. No Claude Code, o *prompt* real geralmente é maior do que a mensagem que você digita: ele inclui também contexto do projeto, arquivos lidos, respostas anteriores, conteúdos de CLAUDE.md, skills carregadas, saídas de comandos e resultados de ferramentas.

Um prompt bom não precisa ser perfeito, mas precisa orientar a tarefa. Em vez de "analisa essa base", é melhor especificar o que fazer, em que arquivo e como reportar.

Arquivo.MD

```
## Tarefa
A coluna data_compra de
@vendas_2025.csv
está com valores em formatos mistos
(DD/MM/AAAA
e AAAA-MM-DD).

## O que fazer
1. Padronize tudo para ISO (AAAA-MM-DD).
2. Reporte quantas linhas foram
   convertidas
   em cada formato.
3. Liste linhas que não conseguiu
   interpretar.
```

O que pensam que é enviado toda vez ao modelo:

Mensagem do usuário

O que realmente é enviado toda vez ao modelo:

Mensagem do usuário

CLAUDE.md

Skills

Histórico de conversa

...

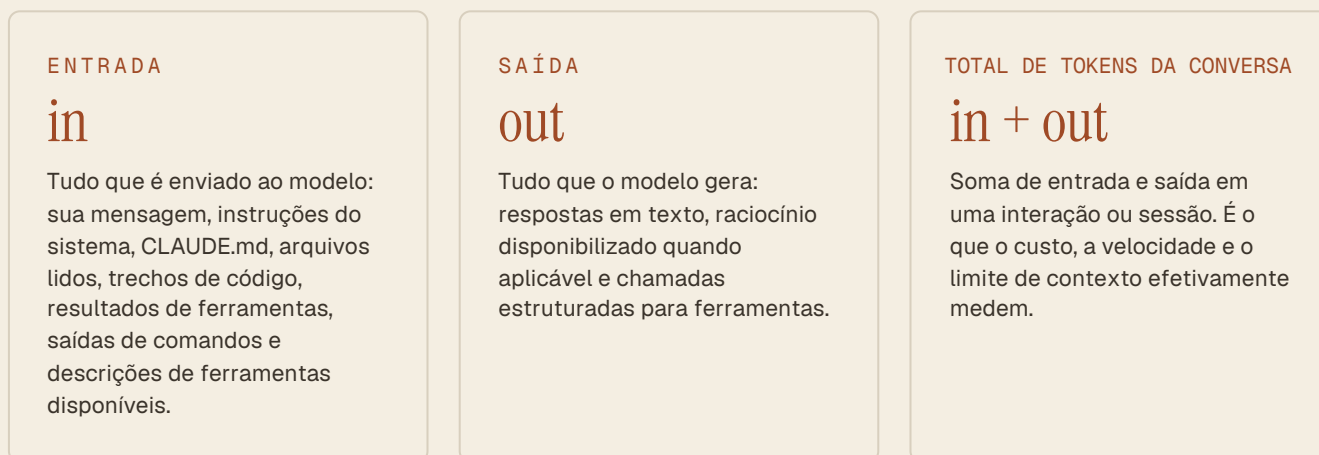
Um arquivo Markdown é um arquivo de texto simples com extensão .md que combina legibilidade com estrutura: títulos, listas e destaques são marcados com símbolos simples como # e **.

Por isso é o formato ideal para escrever prompts e instruções: humanos conseguem ler e editar facilmente, e modelos de linguagem interpretam bem a hierarquia e a organização do conteúdo.

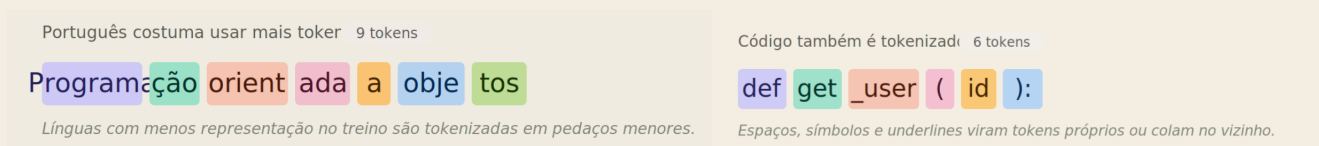
— Tokens, entrada, saída e custo de contexto

O Claude Code usa **tokens** para ler prompts e gerar respostas. Tokens são unidades de texto usadas pelos modelos de linguagem. Um token não é exatamente uma palavra: pode ser uma palavra inteira, parte de uma palavra, pontuação, espaço ou trecho de código. Textos em linguagem natural, código, logs e saídas de terminal são convertidos em tokens antes de serem processados.

Existem três conceitos importantes:



Isso importa porque o custo, a velocidade e o limite de contexto **são influenciados por tokens**. Um arquivo grande lido de uma vez pode consumir milhares de tokens. Um *output* de testes muito longo também. Logs repetitivos, erros irrelevantes e tentativas fracassadas acumuladas na conversa podem **ocupar espaço que seria melhor usado com informações realmente importantes**. Em outras palavras: quanto mais contexto entra na sessão, mais o modelo precisa processar — mas contexto maior não significa necessariamente contexto melhor.



— Modelos da Anthropic e diferenças práticas

A Anthropic organiza seus modelos em famílias com diferentes perfis. É possível selecionar um modelo com /model. Os nomes e limites mudam com o tempo, mas a lógica geral é esta:

OPUS 4.6/4.7	Costuma ser o modelo mais forte para raciocínio complexo. Tende a ser mais caro e/ou mais lento. Janela de contexto: 1M Tokens
SONNET 4.6	Costuma ser o equilíbrio mais usado para programação. Modelo intermediário. Janela de contexto: 1M Tokens
HAIKU 4.5	Costuma ser mais rápido e econômico. Não possui raciocínio tão profundo. Janela de contexto: 200K Tokens

— Janela de contexto

A **janela de contexto** é tudo que a LLM consegue "enxergar" ao mesmo tempo para formular uma resposta. Em um *agent loop* como o Claude Code, essa janela acumula tudo que acontece na sessão — suas mensagens, respostas do Claude, arquivos lidos, *outputs* de comandos, conteúdo de CLAUDE.md, skills carregadas e resultados de ferramentas.

A cada ação nova, mais informação entra. **Nada sai automaticamente** apenas porque deixou de ser importante: o histórico da sessão pode ficar cada vez maior, mesmo quando parte dele já não ajuda mais.

O limite da janela é **medido em tokens e depende do modelo**. Parece muito quando falamos em centenas de milhares de tokens, mas esse espaço **pode esgotar rápido**. Um arquivo grande lido de uma vez pode consumir 10 mil tokens. Um *output* longo de testes pode consumir mais 20 mil. O histórico continua acumulando em cima disso.

ANATOMIA DE UMA JANELA CHEIA

CLAUDE.md + sistema		~4k
Arquivos lidos		~18k
Output de comandos		~24k
Tentativas anteriores		~12k
Logs / ruído acumulado		~9k
Mensagens da sessão		~10k

Quando a janela fica cheia, o problema não é apenas “faltar espaço”. O modelo passa a trabalhar com muito ruído: informações antigas, erros já resolvidos, arquivos que não importam mais e *outputs* repetidos **competem pela atenção com o contexto realmente útil**. Isso pode fazer o Claude esquecer detalhes importantes da tarefa atual, insistir em caminhos antigos, confundir decisões anteriores com requisitos válidos ou responder com menos precisão.

Assim, Claude Code pode compactar o contexto automaticamente. A compactação resume detalhes importantes e remove resultados de ferramentas que já não parecem necessários, liberando espaço. Esse processo é útil, mas tem um risco: **detalhes podem se perder**.

↳ O ponto mais importante é este: **contexto cheio não significa contexto útil**. Se a janela está tomada por logs irrelevantes, tentativas antigas, arquivos que já não importam e *outputs* repetitivos, a qualidade das respostas pode cair. O modelo passa a distribuir atenção por tudo que está ali, inclusive pelo lixo.

Por isso, sessões curtas e focadas, uma por tarefa, costumam funcionar melhor do que uma sessão muito longa tentando resolver tudo de uma vez.

/compact Resume a conversa para liberar espaço na janela de contexto sem começar do zero.

/clear Inicia uma nova conversa com contexto vazio.

02 PRIMEIROS PASSOS

Primeiros passos

— Iniciando com `/init`

Quando você começa a usar o Claude Code em um projeto existente, a primeira necessidade é dar ao Claude uma visão geral do código. Ele precisa entender diretórios, arquivos principais, comandos importantes, estrutura do projeto, padrões de implementação, funções relevantes e estilo de código. Uma forma prática de fazer isso é rodar o comando:

```
● ● ● /init
```

Esse comando cria ou sugere melhorias para um arquivo chamado `CLAUDE.md`. Ao fazer isso, o Claude analisa o projeto e registra informações iniciais que serão úteis nas próximas tarefas: comandos de *build* e teste, convenções encontradas, arquitetura geral e padrões que parecem importantes.

O `/init` não torna o Claude onisciente sobre o projeto. Ele cria um ponto de partida. Depois disso, o arquivo deve ser revisado, editado e enriquecido por você ou pela equipe, especialmente com informações que o Claude não conseguiria descobrir apenas lendo arquivos: regras de negócio, decisões de arquitetura, limitações conhecidas, nomes internos, padrões preferidos e fluxos obrigatórios.

— O que é o CLAUDE.md

O `CLAUDE.md` é um **arquivo de documentação** e contexto que o Claude acessa para orientar seu comportamento dentro do projeto. Ele funciona como uma memória escrita: descreve como o projeto está organizado, quais comandos usar, que padrões seguir e que cuidados tomar. A importância do `CLAUDE.md` está justamente em **evitar que o Claude precise redescobrir tudo a cada tarefa**. Se o projeto tem uma pasta específica para componentes, uma convenção para testes, uma regra de negócio sensível ou um comando correto para rodar validações, isso deve estar registrado ali.

```
## Contexto
Pipeline de ingestão e transformação de
dados de vendas.
Banco principal: PostgreSQL. Orquestração:
Airflow.

## Convenções
- Sempre usar CTEs em vez de subqueries
- Nomes de tabelas em snake_case
- Toda query nova precisa de comentário
  explicando a lógica

## O que evitar
- Nunca deletar dados direto – usar soft
  delete com coluna 'deleted_at'
- Não usar 'SELECT *' em produção

## Testes
Rodar 'pytest tests/' antes de qualquer
PR.
```

As funções principais do CLAUDE.md são:

- ajudar o Claude a entender código, comandos importantes, arquitetura e estilo;
- fornecer direções específicas para o projeto;
- registrar padrões que devem ser seguidos em tarefas futuras;
- orientar onde criar, mover ou modificar arquivos;
- reduzir erros repetidos causados por falta de contexto.

— Tipos de CLAUDE.md e escopo

Existem três tipos de arquivos CLAUDE.md, cada um com um escopo diferente. A regra geral é: **use o escopo mais estreito que ainda resolva o problema.**

COMPARTILHADO

`./CLAUDE.md`

Criado por `/init`.
Compartilhado e versionado com o projeto. Registra contexto geral: arquitetura, comandos de teste, padrões de código, bibliotecas preferidas e fluxos comuns.

LOCAL · SENSÍVEL

`./CLAUDE.local.md`

Não pensado para ser compartilhado. Instruções pessoais ou customizadas para o projeto: URLs de *sandbox*, caminhos da sua máquina, dados que não devem entrar no repositório.
Deve ficar no `.gitignore`.

GLOBAL · MÁQUINA

`~/ .claude/CLAUDE.md`

Vale para *todos* os projetos na sua máquina. Use para preferências globais e instruções permanentes: estilo de comunicação, padrões gerais de segurança, regras amplas do seu trabalho.

Um exemplo de uso: se você quer ajustar o Claude para o seu contexto de negócio e todas as tarefas do seu usuário devem considerar certas regras, o arquivo **global** é o lugar adequado. O cuidado principal é não exagerar — se tudo vira regra global, o contexto fica pesado e genérico demais. Use o arquivo global para instruções realmente permanentes, o CLAUDE.md do projeto para regras do projeto, e skills para procedimentos específicos.

— Editando o CLAUDE.md com instruções claras

Todo CLAUDE.md é modificável. Você pode editar como quiser, usando Markdown. Uma instrução simples pode ser escrita assim:

```
# Sempre que for criar uma nova forma, crie em 3 cores distintas:  
rosa, verde e azul
```

Esse exemplo ensina uma regra de comportamento. Sempre que o Claude precisar criar uma nova forma, ele deve lembrar dessa preferência visual.

Você também pode registrar instruções com contexto de negócio. Por exemplo, imagine que você é analista de dados e quer que o Claude sempre lembre quais bases você usa, quais são as principais variáveis e o que cada uma significa. Em vez de colar essa explicação toda vez, você pode manter um arquivo Markdown com as descrições das tabelas e referenciá-lo dentro do CLAUDE.md.

```
# TODA VEZ QUE PRECISAR ENTENDER QUAL BASE DE DADOS DEVE SER
UTILIZADA, LEIA O ARQUIVO @projeto/bases.md
```

Assim, quando a tarefa envolver escolha ou interpretação de bases de dados, o Claude saberá que deve consultar `@projeto/bases.md` antes de decidir.

— Usando @ para mencionar arquivos

O `@` é usado para mencionar caminhos de arquivos. Isso ajuda o Claude a localizar exatamente o recurso que você quer usar.

```
> @projeto/dados/vendas_2025.csv
> gere um relatório descritivo das colunas numéricas,
> destacando média, mediana, mínimo, máximo e quantidade
> de valores ausentes por coluna.
```

Nesse caso, você não está apenas dizendo "gere um relatório das vendas". Você está apontando o arquivo específico que deve ser usado.

Esse recurso é especialmente útil quando:

- há muitos arquivos com nomes parecidos;
- você quer evitar que o Claude procure no lugar errado;
- a tarefa depende de um arquivo específico;
- você quer forçar o Claude a ler uma documentação ou base antes de agir.

O `@` também funciona com diretórios inteiros (`@projeto/dados/`), com arquivos isolados (`@README.md`) e dentro de outros arquivos `.md`, como o próprio CLAUDE.md, onde você pode amarrar regras a arquivos específicos sem precisar repetir o caminho a cada interação.

↳ Em projetos com muitas pastas e nomes próximos (por exemplo, `vendas_2024.csv`, `vendas_2025.csv`, `vendas_agg.csv`), referenciar com `@` evita uma classe inteira de erros silenciosos em que o modelo escolhe o arquivo errado por afinidade de nome.

03 PLANEJAMENTO E CONTROLE DA SESSÃO

Comandos de planejamento e contexto

— Planning mode e thinking com Esc

O **planning mode** é útil quando você tem uma **sequência de ações ou mudanças a executar**. Em vez de o Claude começar a editar imediatamente, ele usa ferramentas de leitura e análise para entender o projeto e retornar um plano em texto, passo a passo.

Depois de receber o plano, você pode **aprovar, editar ou rejeitar**. Essa etapa é valiosa quando você quer garantir que a direção esteja correta antes de permitir mudanças no código. Use planning mode quando:

- a tarefa envolve várias áreas do projeto;
- é necessário entender a arquitetura antes de alterar;
- há múltiplos passos dependentes;
- o custo de uma mudança errada é alto;
- você quer revisar o caminho antes da execução.

O **"thinking"** ou raciocínio estendido é diferente. Ele é mais útil quando o problema é pontual, mas exige análise cuidadosa: por exemplo, investigar um *bug* específico, entender uma inconsistência ou decidir entre duas soluções pequenas. O planning mode organiza o fluxo de trabalho. O thinking aprofunda o raciocínio sobre uma decisão.

Uma regra prática: **planning mode** é melhor para entendimento amplo, mudanças em sequência e tarefas com vários passos; **thinking** é melhor para *bugs*, decisões pontuais e problemas que exigem raciocínio mais profundo, mas não necessariamente um plano grande.

Planning mode tende a consumir mais tokens, porque analisa mais contexto e produz um plano detalhado. Isso tem custo associado. Use quando o benefício de planejar antes de executar for maior que o custo de contexto.

```
Tarefa: migrar tabela de usuários para novo schema
```

```
Plano
```

1. Ler schema atual em `db/migrations/`
2. Comparar com novo modelo em `docs/schema\_v2.md`
3. Identificar colunas removidas, renomeadas e adicionadas
4. Escrever migration SQL com rollback
5. Atualizar queries afetadas em `src/repositories/`
6. Rodar testes de integração

```
Antes de executar
```

```
Confirmar com o usuário quais tabelas dependentes serão afetadas.
```

— Interromper com `Esc`

Durante uma tarefa, o Claude pode seguir por um caminho errado: procurar arquivos que não existem, interpretar mal uma regra de negócio, repetir um comando incorreto ou insistir em uma solução que você já sabe que não funciona. Quando isso acontece, **interrompa**.

No terminal, pressionar `Esc` permite parar o raciocínio ou a execução e redirecionar. A ideia é tratar o Claude como um colega de trabalho — se ele está indo na direção errada, não espere terminar para corrigir.

Especialmente útil junto com memória persistente: se o Claude comete repetidamente o mesmo erro de caminho, interrompa, corrija e registre no CLAUDE.md.

```
# Quando precisar localizar os
relatórios
financeiros mensais, use sempre
@projeto/financeiro/relatorios.md.
```

A vantagem de interromper cedo é evitar que uma cadeia inteira de tentativas erradas entre na janela de contexto.

— Compactar contexto com `/compact`

Quando a janela de contexto fica cheia, o Claude Code pode compactar a conversa. A compactação resume a sessão, preservando informações-chave e removendo detalhes ou resultados de ferramentas que parecem menos necessários. Você pode acionar isso manualmente com:

```
> /compact
```

A compactação é útil quando você quer continuar na mesma tarefa, mas percebe que há muito histórico acumulado. Ela permite liberar espaço sem começar do zero. O cuidado é que compactar pode perder detalhes. Se há algo muito importante, como uma decisão de arquitetura, uma regra de negócio ou uma exceção técnica, deixe isso explícito antes de compactar. Por exemplo:

— Voltar com `/rewind`

Às vezes, o problema não é o Claude, mas o ambiente. Imagine que você esqueceu de instalar um pacote: o Claude roda testes, encontra erros, tenta corrigir, muda abordagens e só depois percebe que o pacote não estava instalado.

Mesmo resolvido, toda essa volta fica registrada no contexto. Nesses casos:

```
> /rewind
```

Ou pressione `Esc` duas vezes, quando o atalho estiver disponível. O `rewind` permite voltar a um ponto anterior e escolher o que fazer:

- manter contexto valioso (entendimento do projeto);
- remover histórico irrelevante ou distrativo;
- desfazer alterações que deram errado;
- economizar espaço na janela.

O `rewind` não é um "desfazer" — é uma forma de **gestão de sessão**.

```
> /compact Preserve a decisão de que a coorte de clientes  
> premium é definida por  $\geq 12$  meses de contrato + ticket  
> médio  $\geq$  R$ 500, e mantenha o caminho da base  
> @projeto/dados/coorte_premium_parquet
```

Assim, você orienta o resumo a preservar o que realmente importa.

— Começar do zero com `/clear`

Quando você quer mudar para uma tarefa sem relação com a anterior, use:

```
> /clear
```

Esse comando começa uma nova conversa com contexto vazio. Ele é diferente do `/compact`: compactar mantém a sessão, mas resume o histórico; limpar começa do zero. Use `/clear` quando:

- a nova tarefa não tem relação com a anterior;
- o contexto atual está poluído demais;
- você quer evitar que decisões antigas influenciem a próxima tarefa;
- o assunto mudou completamente;
- você quer testar uma abordagem nova sem herdar histórico.

Uma prática recomendada para iniciantes é abrir sessões separadas para tarefas separadas. Em vez de passar horas na mesma sessão misturando *bugs*, refatorações, testes e dúvidas conceituais, prefira criar blocos de trabalho mais curtos e focados.

04 COMANDOS ESSENCIAIS E PERSONALIZADOS

Comandos essenciais e personalizados.

— Comandos essenciais

Os comandos do Claude Code podem ser acessados escrevendo `/` no início da linha. O menu mostra comandos internos, skills, comandos do usuário, comandos de *plugins* e, em alguns casos, comandos vindos de MCP Servers. Para iniciantes, estes são especialmente úteis:

<code>/init</code>	Cria ou melhora o CLAUDE.md do projeto, ajudando o Claude a mapear comandos, estrutura e convenções.
<code>/model</code>	Permite trocar o modelo durante a sessão, quando disponível.
<code>/compact</code>	Resume a conversa para liberar espaço na janela de contexto sem começar do zero.
<code>/clear</code>	Inicia uma nova conversa com contexto vazio.
<code>/rewind</code>	Volta para pontos anteriores, restaura ou resume partes da conversa e dos arquivos.
<code>/context</code>	Mostra uso de contexto e ajuda a identificar o que está ocupando espaço.
<code>/config</code>	Abre preferências e configurações.
<code>/hooks</code>	Mostra hooks configurados e ajuda a verificar eventos disponíveis.
<code>/agents</code> · <code>/agent</code>	Ajuda a criar ou configurar subagentes, dependendo da versão e configuração.
<code>/doctor</code>	Diagnostica problemas comuns de instalação e configuração.

Além de `/`, há outros atalhos úteis: `@` no início de um caminho ajuda a mencionar arquivos; `!` no início pode acionar modo *shell* em alguns fluxos, executando comandos diretamente e adicionando o *output* à sessão; `Shift+Tab` pode alternar modos de permissão, incluindo modo padrão, *auto-accept*, *edits*, *plan mode* e outros modos disponíveis.

— Comandos personalizados como prompts reutilizáveis

Você também pode criar comandos personalizados usando arquivos Markdown. Esses comandos são *prompts* prontos, acessíveis por `/`, que podem ser executados sempre que você quiser repetir a mesma instrução. A ideia é simples: em vez de escrever o mesmo pedido toda vez, você cria um comando, como `/analise_descritiva`, e o Claude insere automaticamente aquele *prompt* estruturado.

Um comando personalizado também pode aceitar argumentos. O marcador `$ARGUMENTS` representa o texto que você passa depois do nome do comando. Exemplo:

```
# /analise_descritiva

Faça uma análise descritiva completa para: $ARGUMENTS

## Convenções
- Trabalhe em Python, usando pandas e seaborn.
- Salve os outputs em `outputs/analise/`.
- Nomeie arquivos como `[base]_[tipo].png` ou `[base]_[tipo].csv`.

## Cobertura mínima
- Tipos das colunas, valores ausentes e únicos.
- Estatísticas (média, mediana, desvio, mín/máx) das numéricas.
- Distribuição (histograma) das 5 principais variáveis numéricas.
- Frequência das principais categóricas.
- Correlação entre numéricas (matriz com heatmap).
- Lista de outliers detectados via IQR.
```

Entrada do usuário:

```
> /analise_descritiva o arquivo @projeto/dados/vendas_2025.csv
```

Nesse caso, o Claude entende que deve executar uma análise descritiva completa para o arquivo `vendas_2025.csv`, seguindo pandas + seaborn, salvando em `outputs/analise/`, respeitando o padrão de nome e cobrindo a checklist obrigatória.

Nas versões atuais do Claude Code, comandos personalizados e skills se aproximaram bastante: ambos podem aparecer como comandos com `/`, e arquivos antigos em `.claude/commands/` continuam funcionando em muitos fluxos. Para iniciantes, a distinção prática é esta: pense no **comando personalizado** como um atalho para um *prompt* bem definido; pense na **skill** como conhecimento ou procedimento especializado que o Claude pode carregar quando for relevante.

05 HOOKS

Hooks.

— O que é um hook

Um **hook** é um **comando executado por evento**. Isso significa que, antes ou depois de uma ação do Claude, um hook pode ser **acionado automaticamente**. Ação, aqui, pode significar ler um arquivo, editar um arquivo, executar um comando, criar algo, rodar uma busca ou usar uma ferramenta. O hook entra no fluxo para garantir que certas ações aconteçam sempre, sem depender do modelo lembrar.

Imagine que, toda vez que o Claude editar um arquivo de dados, você queira **validar automaticamente** o *schema* com uma ferramenta específica. Sem hook, você precisa lembrar de pedir isso manualmente. Com hook, você configura uma vez e a validação roda sozinha sempre que o Claude salva ou altera o arquivo.

A característica mais importante do hook é que ele é **determinístico**. Ele não depende da LLM decidir se deve rodar ou não. O Claude pode nem "saber" que o hook aconteceu. O evento dispara, o comando executa e o resultado pode ou não ser devolvido ao fluxo, dependendo da configuração.

Hooks são úteis quando uma regra precisa ser aplicada de forma confiável. Se algo é importante demais para depender de uma instrução em linguagem natural, provavelmente deve virar hook, permissão ou automação.

— PreToolUse e PostToolUse

Dois tipos de hook são especialmente importantes. Eles diferem só no *quando* — antes ou depois da ação — mas isso muda completamente o que cada um serve para fazer.

ANTES DA AÇÃO

PreToolUse

Acionado **antes** de o Claude usar uma ferramenta. Serve para **bloquear, validar, registrar ou modificar** o comportamento antes da ação acontecer.

Exemplo: impedir que o Claude leia arquivos com informações sensíveis, como `.env`, chaves privadas ou credenciais.

DEPOIS DA AÇÃO

PostToolUse

Acionado **depois** de o Claude usar uma ferramenta. Serve para **registrar** o uso da ferramenta ou gerar *feedback* depois da ação.

Exemplo: toda vez que o Claude editar um arquivo de dados, rodar uma checagem de *schema* e devolver o *feedback* para a sessão.

— Construindo um hook

Para configurar um hook, é necessário definir em que tipo de evento ele será acionado. Em muitos casos, você delimita o hook pelo tipo de *tool call*. No exemplo da validação automática, o hook seria associado a ações de edição, como `EDIT` ou `WRITE`, dependendo da nomenclatura da versão e ferramenta.

Construindo um hook

- 1

Decidir entre um hook
`PreToolUse` ou `PostToolUse`
- 2

Determinar quais *tool calls* você quer observar
- 3

Escrever um comando que vai receber a *tool call*
- 4

Se necessário, o comando deve devolver *feedback* ao Claude

Nomes de ferramentas

Nome	Finalidade
<code>Read</code>	Ler um arquivo
<code>Edit</code> , <code>MultiEdit</code>	Editar um arquivo existente
<code>Write</code>	Criar um arquivo e escrever nele
<code>Bash</code>	Executar um comando
<code>Glob</code>	Encontrar arquivos ou pastas por padrão
<code>Grep</code>	Buscar conteúdo dentro de arquivos
<code>Task</code>	Criar um subagente para uma tarefa específica
<code>WebFetch</code> , <code>WebSearch</code>	Buscar ou abrir uma página específica

Você pode pedir ao próprio Claude Code para listar os eventos e ferramentas disponíveis na sua instalação. Isso é importante porque nomes, eventos e permissões podem mudar entre versões.

— Exemplos de uso de hooks

Hooks podem ser usados em vários fluxos práticos. Em projetos de dados, especialmente, eles ajudam a garantir higiene de pipeline sem depender do modelo lembrar de fazer cada checagem.

FORMATAÇÃO	Formatar arquivos automaticamente depois que o Claude os editar — <code>black</code> , <code>ruff</code> , <code>prettier</code> ou similar.
TESTES	Rodar testes automaticamente quando arquivos mudarem (<code>pytest</code> , <code>dbt test</code> , validações de <i>schema</i>).
CONTROLE DE ACESSO	Bloquear leitura ou edição de arquivos específicos: <code>.env</code> , credenciais, bases sensíveis com PII.
QUALIDADE	Rodar <i>linters</i> , <i>type checkers</i> ou validações de qualidade de dados e fornecer <i>feedback</i> ao Claude.
LOGGING	Rastrear quais arquivos o Claude acessa ou modifica — útil para auditoria em projetos com dados regulados.
VALIDAÇÃO	Verificar convenções de nome, padrões de código ou regras da equipe.

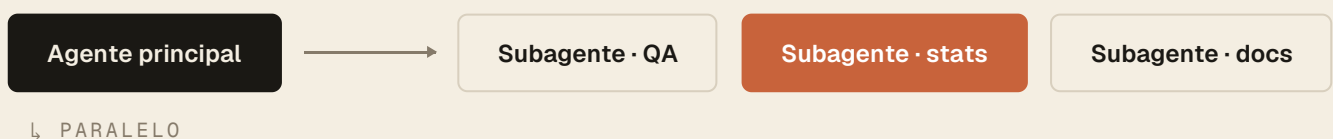
↳ Uma boa regra: se você quer que algo aconteça **sempre**, use hook. Se você só quer orientar o Claude sobre como pensar ou agir, use CLAUDE.md ou skill.

06 SUBAGENTES

Subagentes.

— O que são subagentes

Subagentes são agentes especializados, isolados e configurados para trabalhar em tarefas específicas: um para *data quality*, outro para revisão estatística, outro para *feedback* de código de análise, outro para revisão de documentação de modelos, por exemplo.



A principal vantagem é **lidar melhor com janelas de contexto**. Em vez de concentrar tudo em um único modelo generalista dentro da conversa principal, o Claude Code **delega** tarefas para agentes com *prompts* personalizados, ferramentas próprias e **contexto separado**. Cada subagente tem sua própria janela: pode ler muitos logs, explorar arquivos, rodar comandos ou investigar parte do problema sem poluir a conversa principal. Quando termina, devolve um resultado resumido ao orquestrador.

— Como o Claude usa subagentes

Quando você envia um *prompt*, o Claude Code pode **analisar os subagentes disponíveis e decidir se algum deles combina com a tarefa**. Essa decisão normalmente depende da **descrição** do subagente. Por isso, descrições claras são essenciais.

Um subagente pode ser entendido como uma **instância separada do Claude**, invocada programaticamente por uma ferramenta de tarefa dentro do Claude Code. Ele roda de forma isolada e autônoma dentro de um fluxo maior. Um subagente pode:

- ter **contexto próprio**;
- ter ferramentas e permissões próprias;
- executar múltiplos passos internamente;
- usar Bash, ler arquivos, editar ou fazer *commits*, se tiver permissão;
- rodar em paralelo com outros subagentes;
- devolver um resultado estruturado para o orquestrador;
- ajudar o agente principal a continuar sem carregar todo o histórico da investigação.

Isso permite uma **arquitetura em paralelo**. O agente principal coordena. Os subagentes executam partes do trabalho. O resultado volta para que o fluxo continue.

— Criando e configurando subagentes

Para criar um subagente, use o comando disponível na sua versão, como:

```
> /agents
```

Em alguns materiais ou versões, o comando pode aparecer como `/agent`. O importante é acessar o fluxo de criação de agentes. Você pode pedir ao Claude para gerar um subagente ou configurá-lo manualmente. Se pedir para o Claude criar, descreva o papel do agente, por exemplo:

```
# Criar subagente
Crie um subagente especializado em revisar qualidade de bases de dados de vendas.

Ele deve verificar valores ausentes acima de 5% por coluna, tipos inconsistentes, datas em formatos mistos, duplicidades por chave primária declarada e outliers grosseiros via IQR.
```

Durante a configuração, o Claude pode perguntar quais ferramentas o agente deve acessar: apenas leitura, edição, execução de comandos, ou outras. Essa escolha deve ser intencional. Um agente de **feedback**, por exemplo, talvez só precise ler arquivos. Um agente de testes pode precisar ler, executar comandos e talvez editar testes. Um agente de *deploy* provavelmente exigiria permissões muito mais sensíveis.

Depois, o fluxo pode pedir para selecionar modelo e cor para o subagente. A cor ajuda na visualização da sessão. O modelo deve ser escolhido de acordo com a tarefa: um agente que faz revisão profunda pode exigir modelo mais forte; um agente que apenas classifica arquivos pode usar um modelo mais rápido. O Claude cria uma pasta de agentes com nome, descrição e ferramentas permitidas. Uma boa prática é abrir esse arquivo depois, ler o conteúdo e fazer ajustes. **O texto do subagente é o que define seu comportamento real.**

— Quando subagentes funcionam bem

Subagentes funcionam melhor quando as tarefas são **independentes entre si**. Exemplos: rodar *lint*; rodar testes; fazer verificação de qualidade de dados; revisar documentação; procurar arquivos relevantes; analisar uma parte isolada do código; gerar *feedback* sobre uma implementação já pronta. Nesses casos, cada subagente pode fazer sua parte sem precisar saber o que os outros estão fazendo. Isso economiza contexto e permite paralelismo.

— O problema dos subagentes

O problema aparece quando uma tarefa depende do resultado de outra. Como cada subagente tem contexto isolado, ele começa do zero e não tem acesso automático ao que outros agentes fizeram. Isso não significa que tarefas dependentes sejam impossíveis. Significa que você precisa projetar o fluxo de forma sequencial.

Se o resultado do subagente A é necessário para o subagente B, o orquestrador precisa **passar explicitamente** esse resultado para o subagente B. A dependência funciona, mas não acontece sozinha.

Exemplo prático: você usou o Claude normalmente para criar um *pipeline* de tratamento de dados. Depois aciona um subagente especializado em analisar esse *pipeline*. Esse subagente não vai conhecer todo o processo que levou à implementação. Ele não vê automaticamente a conversa anterior, as decisões intermediárias, os erros corrigidos ou as alternativas descartadas. Ele apenas recebe o código e o contexto que foi passado para ele. Isso caracteriza justamente a **janela de contexto isolada**. Portanto:

- subagentes funcionam muito bem para tarefas realmente independentes;
- subagentes podem falhar quando cada passo depende de descobertas do passo anterior;
- dependências precisam ser explicitadas pelo agente orquestrador;
- quanto melhor a descrição e o contexto passado, melhor será o resultado do subagente.

↳ Pense em subagentes como consultores externos: ótimos para um diagnóstico isolado, problemáticos quando a tarefa exige que cada um saiba o que o anterior decidiu. Quando perceber dependência entre passos, é melhor consolidar no agente principal — ou explicitar a entrega de contexto entre subagentes.

07 SKILLS

Skills.

— O que é uma skill

Uma **skill** é um arquivo Markdown que ensina o Claude a fazer alguma coisa bem. Ela pode conter instruções, procedimentos, *checklists*, padrões, exemplos e arquivos de apoio. Em vez de colar o mesmo conhecimento em todo *prompt*, você cria a skill uma vez e o Claude pode usá-la quando for relevante. Pense em uma skill como **conhecimento especializado sob demanda**. Exemplos:

- regras de identidade visual da empresa para criar apresentações;
- cores oficiais, tipografia e estilo de *slides*;
- convenções de *commit*;
- padrão de descrição de *pull request*;
- processo de revisão de segurança;
- *checklist* para análise de dados;
- forma correta de gerar relatórios para um time específico.

A skill não é apenas um atalho. **Ela ensina um modo de executar uma tarefa com qualidade.**

— Como o Claude decide usar uma skill

A parte mais importante de uma skill é a **descrição**. O Claude lê o seu *prompt*, compara com as descrições das skills disponíveis e decide quais parecem relevantes. Uma boa descrição precisa dizer duas coisas: **o que a skill faz** e **quando o Claude deve usá-la**. Exemplo de descrição boa:

```
name: analise-descritiva
description: Roda uma análise descritiva padrão sobre uma base
de dados tabular. Use quando o usuário pedir descritiva, EDA,
exploração inicial, perfilamento de base, profiling, distribuição
de variáveis ou estatísticas básicas de uma base.
```

Essa descrição funciona porque deixa claro o propósito e o gatilho. Se o usuário pedir "faz uma descritiva dessa base" ou "explora esse CSV pra mim", o Claude tem uma chance maior de ativar a skill certa. Campos úteis em skills podem incluir:

- *allowed tools*: ferramentas que a skill pode usar, como `read`, `grep`, `glob` ou `bash`;
- *model*: qual modelo usar quando a skill for executada, como Sonnet, Opus ou outro disponível;
- instruções internas;
- arquivos de referência;
- *scripts* auxiliares;
- exemplos de saída esperada.

— Diferença entre CLAUDE.md, skill e slash command

Esses três recursos se parecem, mas têm funções diferentes.

CLAUDE.md	Roda ou é carregado como contexto persistente nas sessões. Use para fatos, regras e padrões que o Claude precisa considerar com frequência: comandos do projeto, arquitetura, estilo de código, bibliotecas preferidas e regras gerais.
SKILL	Carrega sob demanda quando dá <i>match</i> com a tarefa. Use para conhecimento especializado ou procedimentos que não precisam ocupar contexto o tempo todo. Uma skill não precisa executar nada sozinha; ela fornece instruções relevantes para o Claude executar melhor quando a situação exige.
SLASH COMMAND	É um gatilho manual . Você chama algo como <code>/review</code> e ele executa um <i>prompt</i> específico. O foco é disparar uma ação com uma instrução definida.

Uma forma simples de memorizar: **CLAUDE.md** é contexto persistente; **skill** é conhecimento especializado sob demanda; **slash command** é um gatilho para uma ação repetível.

Na prática, skills e comandos personalizados ficaram mais próximos nas versões atuais do Claude Code, porque skills também podem ser chamadas com `/`. Ainda assim, a distinção conceitual ajuda: comandos são atalhos de ação; skills são pacotes de conhecimento e procedimento.

— Como criar uma skill

Para criar uma skill, primeiro crie um diretório dentro da pasta de skills do Claude. O nome da pasta será o nome da skill.

```
~/ .claude/
├── skills/
│   └── analise-descritiva/
│       └── SKILL.md
```

Depois, crie um arquivo chamado exatamente `SKILL.md`. O nome precisa ser `SKILL.md`. Se o arquivo tiver outro nome, a skill pode não funcionar.

```
---
name: analise-descritiva
```

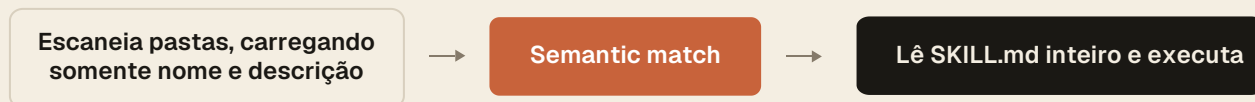
```
description: Roda uma análise descritiva padrão sobre uma base tabular. Use quando o usuário pedir descritiva, EDA, exploração inicial ou perfilamento de base
```

```
---
Ao executar uma análise descritiva:
```

1. Liste tipos das colunas e contagem de valores ausentes.
2. Calcule estatísticas (média, mediana, desvio, mín/máx) das numéricas.
3. Gere histograma das 5 numéricas com maior variância.
4. Calcule frequência das principais categóricas.
5. Detecte outliers via IQR e liste-os por coluna.
6. Salve cada saída em `'outputs/analise/'` com nome `'[base]_[tipo]'`.

— Ciclo de vida da skill

Quando o Claude Code inicia, ele escaneia locais onde skills podem estar instaladas. Um detalhe importante: ele **não carrega o conteúdo completo** de todas as skills imediatamente — começa carregando apenas nome e descrição. Isso economiza tokens.



A vantagem é justamente ser um processo feito sob demanda, economizando tokens

Quando você envia uma solicitação, o Claude compara seu pedido com as descrições das skills disponíveis. Por exemplo, "explica o que esse conjunto de variáveis revela sobre churn" pode combinar com uma skill descrita como "explicar relações entre variáveis com diagramas e tabelas". Quando uma skill relevante é encontrada, ela pode ser carregada automaticamente ou pedir confirmação. Depois disso, o Claude lê o `SKILL.md` completo e segue as instruções. Essa dinâmica permite manter muito conhecimento disponível sem jogar tudo na janela de contexto desde o início.

— Skills grandes e *progressive disclosure*

Às vezes uma skill fica grande demais — muitas instruções, referências, documentos, *scripts*, exemplos. Se tudo for carregado de uma vez, a janela é consumida rapidamente.

A solução é **progressive disclosure**: em vez de colocar tudo dentro do `SKILL.md`, você cria um arquivo principal curto que funciona como índice e referencia arquivos auxiliares apenas quando necessário. Uma skill de modelagem estatística pode mencionar um `guia-modelos.md`, mas instruir o Claude a lê-lo só quando o usuário perguntar sobre escolha entre regressão, logística ou árvore.

REGRA PRÁTICA

Mantenha o SKILL.md abaixo de 500 linhas.

Se estiver passando disso, divida em arquivos de referência separados. É como ter um sumário na janela de contexto, e não o livro inteiro.

— Quando a skill não é usada como esperado

Se o Claude não usa uma skill quando você esperava, a causa mais provável é a **descrição**. O Claude usa *semantic matching*: compara o significado do seu *prompt* com a descrição da skill. Se ela for vaga, curta demais, específica demais ou não mencionar o caso de uso correto, o *match* pode falhar. Verifique se:

- o arquivo se chama exatamente `SKILL.md`;
- está na pasta correta;
- o nome da pasta corresponde ao nome desejado;
- a descrição explica o que a skill faz e quando usar;
- o caso de uso do seu *prompt* aparece na descrição;
- a skill não está ampla demais (ativa em momentos errados);
- nem estreita demais (não ativa quando deveria).

Na maioria dos casos, **melhorar a descrição resolve**.

08 MCP SERVER

MCP Server

— O que é MCP Server

MCP significa *Model Context Protocol*. É uma **forma de conectar o Claude a ferramentas externas**, como GitHub, Notion, Slack, bancos de dados, ferramentas internas, APIs e sistemas de monitoramento. Sem MCP Servers, muitas vezes você precisa copiar e colar informações dessas ferramentas no chat. Com MCP Server, o Claude pode acessar a ferramenta diretamente, dentro das permissões configuradas. Pense no MCP Server como uma **ponte** entre o Claude Code e outros sistemas — ele dá ao Claude a capacidade de abrir outros "programas" e usá-los enquanto trabalha.

Exemplos de coisas que um MCP Server pode permitir:

- ler *issues* no GitHub ou Jira;
- consultar documentação no Notion;
- buscar mensagens no Slack;
- consultar dados em PostgreSQL;
- abrir *designs* do Figma;
- acessar logs ou métricas em ferramentas de monitoramento;
- interagir com um navegador;
- chamar APIs internas.

O ponto central é reduzir trabalho manual. Em vez de você trazer contexto externo para o Claude, o Claude busca o contexto na fonte.

— Exemplo: Playwright MCP Server

Um dos MCP Servers populares é o **Playwright**, que permite ao Claude controlar um navegador. Isso abre possibilidades fortes para desenvolvimento web. Um fluxo prático seria pedir ao Claude para:

1. abrir um navegador e navegar até a aplicação;
2. gerar um componente de teste;
3. analisar o estilo visual e a qualidade do código;
4. atualizar o *prompt* de geração com base no que observou;
5. testar o *prompt* melhorado com um novo componente.

Esse tipo de fluxo é difícil de fazer apenas com texto, porque envolve olhar para uma interface, interagir com páginas, comparar visualmente e ajustar código. Com MCP Server e ferramentas de navegador, o Claude pode participar de um ciclo mais próximo do desenvolvimento real.

09

COMO ESCOLHER O RECURSO CERTO

Como escolher o recurso certo.

— Mapa de decisão para iniciantes

Quando você começa a usar Claude Code, é comum confundir CLAUDE.md, skills, hooks, subagentes, MCP Servers e comandos. Todos parecem "instruções para o Claude", mas cada um resolve um problema diferente. Use a tabela abaixo como referência rápida.

RECURSO	USE QUANDO...	EXEMPLO
CLAUDE.md	A informação deve estar presente em praticamente toda sessão.	Comandos do projeto, arquitetura, convenções de teste, regras de negócio permanentes.
CLAUDE.local.md	A informação é pessoal, local ou não deve ser compartilhada.	URLs de <i>sandbox</i> , caminhos locais, preferências individuais.
~/.claude/CLAUDE.md	A instrução vale para todos os seus projetos.	Preferências globais de estilo, regras gerais de segurança, contexto fixo do seu trabalho.
Skill	O Claude precisa aprender um procedimento especializado sob demanda.	Guia de identidade visual, padrão de PR, <i>checklist</i> de segurança, análise descritiva.
Slash command	Você quer chamar manualmente um <i>prompt</i> repetível.	<code>/write_tests</code> , <code>/review</code> , <code>/deploy_checklist</code> .
Hook	Algo precisa acontecer sempre, de forma determinística.	Rodar <i>formatter</i> após edição, bloquear <i>secrets</i> , executar <i>lint</i> , registrar logs.
Subagente	A tarefa é especializada, independente ou pode poluir o contexto principal.	Revisão de segurança, execução de testes, pesquisa em arquivos, análise paralela.
MCP Server	O Claude precisa acessar uma ferramenta ou fonte externa.	GitHub, Notion, Slack, banco de dados, Playwright, Figma.
Planning mode	A tarefa tem muitos passos ou risco de alteração errada.	Refatoração, nova <i>feature</i> , mudança arquitetural.
Thinking	O problema é pontual, mas exige raciocínio profundo.	<i>Bug</i> difícil, decisão técnica específica, análise de causa raiz.

— Matriz comparativa: CLAUDE.md, Slash, Hook, Agent, Skill

A tabela abaixo cruza cada recurso com os outros para deixar explícita a diferença prática entre eles. Em cada célula, a frase descreve o papel do recurso da linha em relação ao da coluna; o pequeno rótulo em mono ao final resume o contraste.

	CLAUDE.md	Slash Cmd	Hook	Agent	Skill
CLAUDE.md	—	CLAUDE.md é sempre ativo. Slash Cmd só age quando chamado. <i>sempre-on vs manual</i>	CLAUDE.md define padrões. Hook executa ações por eventos. <i>padrão vs reação</i>	CLAUDE.md guia a conversa atual. Agent roda em contexto isolado. <i>contexto atual vs isolado</i>	CLAUDE.md carrega sempre. Skill carrega só quando o tema é relevante. <i>sempre vs sob demanda</i>
Slash Cmd	SC dispara uma tarefa pontual. CLAUDE.md é regra permanente. <i>pontual vs permanente</i>	—	SC: você pede. Hook: o sistema detecta e age sozinho. <i>manual vs automático</i>	SC é instrução estática. Agent tem autonomia e contexto próprio. <i>instrução vs executor</i>	SC dispara ação. Skill orienta como executar essa ação bem. <i>gatilho vs guia</i>
Hook	Hook roda código ao detectar evento. CLAUDE.md guia o raciocínio. <i>execução vs raciocínio</i>	Hook age por evento sem você pedir. SC age porque você pediu. <i>evento vs pedido</i>	—	Hook faz ação simples e pontual. Agent executa tarefa complexa. <i>pontual vs complexo</i>	Hook é acionado por evento. Skill é acionada pelo que você pede. <i>evento vs requisição</i>
Agent	Agent tem contexto isolado. CLAUDE.md existe só na conversa principal. <i>isolado vs principal</i>	Agent é autônomo e toma decisões. SC é <i>prompt</i> sem autonomia. <i>autônomo vs estático</i>	Agent executa trabalho complexo. Hook faz efeito colateral simples. <i>complexo vs colateral</i>	—	Agent executa em contexto isolado. Skill enriquece o atual. <i>isolado vs enriquece</i>
Skill	Skill carrega sob demanda. CLAUDE.md carrega sempre em toda conversa. <i>demanda vs sempre</i>	Skill orienta <i>como</i> fazer. SC define <i>o que e quando</i> fazer. <i>como vs o quê</i>	Skill ativa pelo que você pede. Hook ativa pelo que acontece. <i>requisição vs evento</i>	Skill adiciona conhecimento ao contexto. Agent roda fora dele. <i>conhecimento vs execução</i>	—

— Boas práticas finais

Para usar Claude Code bem, não basta pedir "faça isso". O ideal é criar um ambiente onde o Claude tenha contexto útil, ferramentas corretas e limites claros. Algumas boas práticas para iniciantes:

- Comece projetos existentes com `/init`.
- Revise o CLAUDE.md gerado e acrescente regras que o Claude não descobriria sozinho.
- Use `@` para indicar arquivos e reduzir ambiguidade.
- Coloque regras permanentes no CLAUDE.md, não em *prompts* soltos.
- Use skills para procedimentos especializados que não precisam estar sempre no contexto.
- Use hooks quando uma regra precisa ser cumprida sempre.
- Use planning mode antes de mudanças grandes.
- Use subagentes para tarefas independentes que poderiam poluir o contexto principal.
- Use MCP Server quando o contexto necessário está fora do repositório.
- Use `/compact` para continuar com menos ruído.
- Use `/rewind` para descartar caminhos errados.
- Use `/clear` para começar uma tarefa sem herdar contexto antigo.
- Mantenha sessões curtas e focadas.

↳ A ideia central é **preservar contexto útil e reduzir ruído**. O Claude Code funciona melhor quando sabe o que deve fazer, tem acesso ao que precisa ler, consegue verificar o resultado e não carrega histórico irrelevante.

— Referências de estudo

[Documentação \(como instalar\)](#)

[Claude Code 101 \(Anthropic\)](#)

[Claude Code in action \(Anthropic\)](#)

[Introduction to agent skills \(Anthropic\)](#)

[Introduction to subagents \(Anthropic\)](#)

[Code w/ Claude Developer Conference](#)

[Como eu uso o Claude Code como analista de dados \(10 casos de uso reais\)](#)

[CLAUDE CODE SKILLS | Full Classroom](#)

[Claude Code - Tutorial Completo \(para Iniciantes\)](#)